Implementation Document

for

Swifty

Version 1.0

Prepared by

Group #8 **Group Name:** Interstellar

Harsh Malgatte	240611	mharsh24@iitk.ac.in
Pulkit Biyani	240814	pulkitj24@iitk.ac.in
Diya H Patel	240739	diyahp24@iitk.ac.in
Vanshika Agrawal	241140	vanshikaag24@iitk.ac.in
Sreeja Srinivasan	241041	sreejas24@iitk.ac.in
Shruti Sahu	241005	shrutisahu24@iitk.ac.in
Saurya Singh	240950	sauryas24@iitk.ac.in
Vasu Goyal	241148	vasug24@iitk.ac.in
Divya Prakash Pandey	240369	divyap24@iitk.ac.in
Rishitesh Kesri	240871	rishitesh24@iitk.ac.in

Course: CS253

Mentor TA: Shaikh Imran

Date: 6 February 2026

CONTENTS II
REVISIONS II

1 IMPLEMENTATION DETAILS 1

2 CODEBASE 2

3 COMPLETENESS 3

APPENDIX A - GROUP LOG 4

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Vanshika, Shruti, Diya, Harsh, Sreeja, Vasu, Saurya, Rishitesh, Divya, Pulkit.	Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded.	00/00/00

1 Implementation Details

1) Frontend:

Using **HTML, CSS, and JavaScript** for the frontend offers universal browser support and seamless integration with any backend.

- **Tailwind CSS:** Provides a utility-first approach that allows for rapid UI development and easy customization of themes.
- **Vanilla JavaScript:** We opted for native JavaScript to handle DOM manipulation and asynchronous fetch requests. This avoids the complexity of heavy frameworks while maintaining high execution speed.
- **Google Fonts & Material Icons:** Used to ensure high readability and a consistent visual language across the dashboard and booking forms.

2) Backend:

For the backend of our web-app, we are using **FastAPI**. This choice was driven by the following advantages:

- **Speed and Performance:** FastAPI is one of the fastest Python frameworks available, rivaling Node.js and Go, thanks to its foundation on Starlette and Pydantic.
- **Asynchronous Programming:** It natively supports async/await, allowing the server to handle multiple concurrent requests (like simultaneous venue bookings) efficiently.
- **Automatic Documentation:** It automatically generates interactive API documentation (Swagger UI), which significantly sped up our testing and frontend integration.
- **Developer Productivity:** The framework is minimalist and intuitive, allowing us to build a robust API with significantly less code than traditional frameworks like Django.

Additionally, to manage our application structure and security, we used several key libraries:

- **SQLAlchemy:** A powerful Object-Relational Mapper (ORM) that allows us to interact with the database using Pythonic classes instead of raw SQL, ensuring better maintainability.
- **PyJWT:** Implemented for JSON Web Token-based authentication, allowing for secure, stateless user sessions.
- **Passlib (Bcrypt):** Used to securely hash and salt user passwords, ensuring that sensitive user data is never stored in plain text.

3) Databases:

For the database layer, we have used **PostgresSQL** managed via the **SQLAlchemy ORM**. PostgreSQL offers several marked advantages for this organization system:

- **Relational Integrity:** A club management system requires strict relationships between users, club roles, and submitted requests. Using a relational database ensures that these constraints are enforced, preventing data inconsistency.

- **ACID Compliance:**PostgresSQL is fully ACID-compliant. This is vital for our program to ensure that approvals and financial entries are processed reliably, guaranteeing that once an authority approves a request, the state is saved permanently and accurately.
- **Efficiency and Speed:** For an application of this scale, PostgresSQL provides lightning-fast read/write operations because it eliminates the network overhead typically found between an application and a separate database server.
- **SQLAlchemy ORM Integration:** By using an Object-Relational Mapper, we can interact with our database using Python classes. This makes our code more maintainable and allows us to easily switch to a more heavy-duty database in the future if the application scales to handle more institute-wide activity.

2 Codebase

Github Repository : https://github.com/sauryas24/swifty

Code structure

The project is divided into two main parts-the codebase and the databases-following the **Model-View-Controller (MVC)** architectural pattern. This separation ensures a clear distinction between data management, user interface, and control logic.

Code Base

The server directory contains the following major files/folders:

- *models.py file contains the schema for all the databases to create the models required for making calls to the database*
- *database.py file file contains the code to connect the database to the application*
- *main.py file that acts as a server and handles all the requests that are made from the frontend sends*

The project is divided into three primary directories based on a decoupled architecture format: the backend, the frontend, and an isolated database layer.

The entire organisational structure is as follows:

The structure of the backend is as follows:

```
backend/
├── requirements.txt          Python dependencies
├── app/      core            application package
├── static/   assets          uploaded file storage
└── tests/    testing         DB seeding & integration checks
```

```
app/      core
├── __init__.py
├── .env                  environment secrets
├── main.py               FastAPI entry point
├── database.py           DB session & engine
├── models.py             SQLAlchemy ORM models
├── schemas.py            Pydantic schemas
├── routers/  API routes  see separate tree
└── utils/    helpers     see separate tree

static/   assets
├── mou_documents/        stored MoU files
└── receipts/             finance receipts

tests/    testing
├── check_db.py           database health check
├── seed_db.py            seed initial data
├── seed_announcements.py seed announcement data
├── test_email.py         email service test
└── test_integration.py   end-to-end tes
```

Organisation of routers/ and utils/.

```
routers/  API routes   one file per feature domain
    ├── __init__.py
    ├── auth.py          user authentication & sessions
    ├── otp.py           one-time password verification
    ├── announcements.py  club announcements
    ├── approvals.py     approval workflow
    ├── calendar.py      event scheduling
    ├── finances.py      financial operations
    ├── MoU.py           memorandum of understanding
    ├── permission.py    role-based access control
    ├── requests.py      request management
    └── venues.py        venue management

utils/  helpers   shared utilities used across routers
    ├── __init__.py
    ├── .env             local secrets
    ├── email_service.py  email sending logic
    └── security.py      JWT tokens & password hashing
```

The structure of the frontend is as follows:

```
frontend/ [html]

  — authority
  ├── Authority_club.html          [club mgmt]
  ├── Authority_dashboard.html     [dashboard]
  ├── authority_login.html         [auth]
  — coordinator
  ├── Coordinator_homepage.html    [home]
  ├── coordinator_login.html       [auth]
  — finance & docs
  ├── Finances.html                [finance]  financial records
  ├── Mou.html                     [docs]  memorandum of understanding
  ├── permissionletter.html        [docs]  permission letter generator
  — public & shared
  ├── Announcements.html           [public]  notice board
  ├── otp.html                     [auth]  OTP verification
  ├── past requests.html           [history]  request archive
  ├── publiccalender.html          [public]  event calendar
  ├── select_role_login.html       [auth]  role selector
  ├── Venue_booking.html           [booking]  venue reservation
```

The structure of the database is as follows:

```
database/ [db]

  ├── main.py      [entry point]  app initialisation
  ├── models.py    [ORM]  SQLAlchemy ORM models
  ├── schemas.py   [schema]  Pydantic schemas
```

The database is a single **PostgresSQL** file managed by SQLAlchemy. The tables and their fields are as follows:

```
users/   user accounts
├── id                    primary key, auto-increment
├── username              club or authority name
├── email_id              unique login identifier
├── password              bcrypt hashed password
├── role                  coordinator or authority
└── associate_council     linked council name

rooms/   venue inventory
├── id                    primary key
├── name                  unique room identifier
└── capacity              maximum occupancy
```

```
venue_bookings/   venue requests
├── id                    primary key
├── date / time           event date and time slot
├── room_id               FK → rooms.id
├── event_title           name of the event
├── event_type            e.g. workshop, concert
├── expected_attendees    estimated head count
├── description           additional event details
├── permission_letter_id  approved PL reference
├── status                tracks approval stage
└── comments              rejection reason if any

clubs/   finance tracking
├── id                    primary key
├── name                  unique club name
├── total_allocated       annual budget in ₹
└── total_spent           running expenditure

transactions/   bill receipts
├── id                    primary key
├── club_id               FK → clubs.id
├── amount                transaction value in ₹
├── description           purpose of expense
├── receipt_url           path to uploaded PDF
└── timestamp             auto-set at creation

permission_letters/   event approvals
├── id                    primary key
├── event_name            name of proposed event
├── date / time           requested event slot
├── reason                purpose / justification
├── club_id               FK → users.id
├── status                approval pipeline stage
├── comments              rejection reason if any
└── generated_id          PL-2026-XXXX on approval
```

```
├── mou_requests/   MoU pipeline
    ├── id                    primary key
    ├── coordinator_id        FK → users.id
    ├── organization_name     external partner
    ├── purpose               scope of agreement
    ├── document_url          uploaded MoU file path
    ├── status                approval pipeline stage
    ├── comments              rejection reason if any
    └── created_at            auto-set timestamp

└── otps/   OTP verification
    ├── id                    primary key
    ├── email_id              one active OTP per email
    ├── otp_code              6-digit verification code
    └── expires_at            ISO string expiry time
```

API Endpoints

The following is a summary of all the possible API calls that are present in the application

1. Authentication

a) Login

```
URL: /auth/login

Method: POST

Request Body (JSON): {

"email_id": "string",

"password": "string" }

Response: Verifies the user's password and generates otp.

Status:{
     If successful: {
         200_OK (Password verified, OTP sent)
             }
     else: {
         401_Unauthorized (Incorrect email or password)
         422_Validation Error (Missing email or password in request)
     }
}
```

b) OTP verification

```
URL: /auth/login/verify

Method: POST

Request Body (JSON): {

"email_id": "string",

"otp_code": "string"

}

Response: Verifies the OTP against the database.
Status: {
     If successful: {
         200_OK (OTP Verified, Token Issued)

           }
```

```
    else: {
        400_Bad Request (Invalid OTP code or OTP has expired)
        404_Not Found (No OTP was requested for this email)
        422_Validation Error (Missing email or OTP code in request)
    }
}
```

2. Public Calendar

a) Get Public Calendar events

URL:/calendar/events

Method: GET

Request Body: None

Response: Fetches all upcoming approved events for the Public Calendar UI.

```
Status:{

      If successful:{

      200_OK (Events retrieved successfully)

            }

      }
```

3. Announcements

a) Publish Announcement

URL:/announcements/publish

Method: POST

Request Body (JSON):{ "heading": "string",

"message": "string",

"target_clubs": ["string"] }

Response: Allows administrative authorities to publish announcements and emails targets
.

Status:

```
{
    If successful: {
        200_OK (Announcement published and emails sent)
    }
    else: {
        401_Unauthorized (User is not logged in)
        403_Forbidden (Only administrative authorities can publish
announcements)
        422_Validation Error (Missing required fields)
    }
}
```

b) View My Announcements

URL: /announcements/my-announcements

Method: GET

Request Body (JSON): None

Response: Club coordinators can view announcements relevant to their club. Authorities can view ALL announcements on their dashboard.

Status:

```
{
    If successful: {
        200_OK (Announcements retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
    }
}
```

4. Permission Letter

a) Submit Permission Letter

URL:/permission/submit

Method: POST

Request Body (JSON):{ "event_name": "string", "date": "string", "time": "string", "reason": "string" }

Response: Takes form data from the frontend and creates a new permission letter.

```
Status:{
    If successful: {
```

```
        200_OK (Permission letter created)
    }
    else: {
        401_Unauthorized (User is not logged in)
        403_Forbidden (Only coordinators can submit permission
letters.)
        422_Validation Error (Missing required fields)
    }
}
```

b) Get Single Permission Letter

URL:/permission/{letter_id}

Method: GET

Request Body: None

Response: Fetches the full details of a specific permission letter by its ID.

```
Status:{

    If successful: {
        200_OK (Permission letter retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        404_Not Found (Permission letter not found)
        422_Validation Error (Invalid letter_id format)
    }
}
```

5. Venue Booking

a) Get Venue Availability

URL: /venues/availability?date={string}&time={string}

Method: GET

Request Body: None

Response: Checks which rooms are available for a given date and time slot.

```
Status:{
```

```
    If successful: {
        200_OK (Availability retrieved)
    }
}
```

b) Create Venue Booking

URL: `/api/venues/book`

Method: POST

```
Request Body (JSON):{ "date": "string", "time": "string", "room_id":
"integer", "event_title": "string", "event_type": "string",
"expected_attendees": "integer", "description": "string",
"permission_letter_id": "string" }

Response: Creates a new venue booking. Enforces Role-Based Access
Control (RBAC) and validates that the permission letter belongs to
the user.

Status:{
    If successful: {
        201_Created (Booking request submitted successfully)
    }
    else: {
        400_Bad Request (Permission letter has not been fully
approved yet)
        401_Unauthorized (User is not logged in)
        403_Forbidden (User is not a coordinator OR letter belongs to
another club)
        404_Not Found (Permission letter not found or invalid ID)
        409_Conflict (Room is already booked for this date and time)
        422_Validation Error (Missing required fields)
    }
}
```

6. MOU

a) Submit MoU Request

URL: `/mou/submit`

Method: POST

```
Request Body(multipart/form-data):{ organization_name: "string",
purpose: "string", document: [File Upload]}
```

```
Response: Allows a coordinator to submit a new MoU request and saves
the attached document.

Status:
{
    If successful: {
        200_OK (MoU request submitted successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        403_Forbidden (Only coordinators can submit MoU requests)
        422_Validation Error (Missing required form fields or file)
    }
}
```

b) Get Specific MOU

```
URL: /api/mou/{mou_id}

Method: GET

Request Body: None

Response: Fetches the details of a specific MoU by its ID.

Status:{

    If successful: {
        200_OK (MoU retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        404_Not Found (MoU not found)
        422_Validation Error (Invalid mou_id format)
    }
}
```

7. Approvals System

a) Process Venue Booking Approval

```
URL: /approvals/venue/{booking_id}/process

Method: PUT

Request Body (JSON):{ "action": "string", "message": "string",
"otp_code": "string" }
```

```
Response: Processes the approval or rejection of a venue booking
request. Validates the OTP for approvals, advances the request
through the pipeline, and triggers notification emails.

Status:{
    If successful: {
        200_OK (Venue request processed successfully)
    }
    else: {
        400_Bad Request (Invalid action, missing/invalid OTP, or
already processed)
        401_Unauthorized (User is not logged in)
        403_Forbidden (User lacks the required role for this pipeline
stage)
        404_Not Found (Venue request not found)
        422_Validation Error (Missing required fields)
    }
}
```

b) Process MOU Approval

```
URL:/approvals/mou/{mou_id}/process

Method: PUT

Request Body (JSON):{ "action": "string", "message": "string",
"otp_code": "string" }

Response: Processes the approval or rejection of an MoU request.
Validates the OTP for approvals, advances the request through the
pipeline, and triggers notification emails.

Status:{
    If successful: {
        200_OK (MoU request processed successfully)
    }
    else: {
        400_Bad Request (Invalid action, missing/invalid OTP, or
already processed)
        401_Unauthorized (User is not logged in)
        403_Forbidden (User lacks the required role for this pipeline
stage)
        404_Not Found (MoU request not found)
        422_Validation Error (Missing required fields)
    }
}
```

c) Process Permission Letter Approval

```
URL: /api/approvals/permission/{letter_id}/process
```

```
Method: PUT

Request Body (JSON):{ "action": "string", "message": "string ",
"otp_code": "string" }

Response: Processes the approval or rejection of a permission letter.
Validates the OTP, advances the pipeline, and generates a unique
Official Permission Letter ID upon final approval.

Status:{
    If successful: {
        200_OK (Permission letter processed successfully)
    }
    else: {
        400_Bad Request (Invalid action, missing/invalid OTP, or
already processed)
        401_Unauthorized (User is not logged in)
        403_Forbidden (User lacks the required role for this pipeline
stage)
        404_Not Found (Permission letter not found)
        422_Validation Error (Missing required fields)
    }
}
```

d) Lookup Permission Letter by Generated ID

```
URL: /approvals/permission/lookup/{generated_id}

Method: GET

Request Body: None

Response: Look up a fully approved Permission Letter by its unique
generated ID.

Status:{

    If successful: {
        200_OK (Permission letter retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        404_Not Found (No approved permission letter found with this
ID)
        422_Validation Error (Invalid ID format)
    }
}
```

8. Past Requests

a) Get All User Requests (Coordinator Dashboard)

```
URL: /requests/all

Method: GET

Request Body: None

Response: Fetches all MoUs, Permissions, and Venues for the logged-in
coordinator.

Status:{

    If successful: {
        200_OK (Records retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        403_Forbidden (Only coordinators can view this dashboard)
    }
}
```

b) Get Pending Requests for Authority

```
URL: /api/requests/pending

Method: GET

Request Body: None

Response: DASHBOARD: All Clubs, strictly currently Pending this
authority.

Status:{

    If successful: {
        200_OK (Pending records retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        403_Forbidden (Not authorized)
    }
}
```

c) Get History for Authority

URL: /requests/history

Method: GET

Request Body: None

Response: DASHBOARD: All Clubs, strictly Processed History of this authority.

```
Status:{

    If successful: {
        200_OK (Historical records retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        403_Forbidden (Not authorized)
    }
}
```

d) Get Club Requests for Authority

URL: /requests/club/{club_id}

Method: GET

Request Body: None

Response: CLUB PAGE: Single Club, strictly relevant (Pending OR History) to this authority.

```
Status:{

    If successful: {
        200_OK (Club specific records retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        403_Forbidden (Not authorized)
        404_Not Found (Club not found)
        422_Validation Error (Invalid club_id format)
    }
}
```

9. OTP Notifications

a) Send OTP (Login/Registration)

URL:/otp/send

Method: POST

Request Body (JSON):{ "email_id": "string" }

Response: Generates a 6-digit verification code, sets a 5-minute expiration, and emails it to the user.

```
Status:{
    If successful: {
        200_OK (OTP successfully sent)
    }
    else: {
        422_Validation Error (Missing required fields)
        500_Internal Server Error (Failed to send the email)
    }
}
```

b) Verify OTP (Login/Registration)

URL: /otp/verify

Method: POST

Request Body (JSON):{ "email_id": "string", "otp_code": "string" }

Response:Used ONLY for Login/Registration. Verifies the code matches and has not expired.

```
Status:{
    If successful: {
        200_OK (Email successfully verified)
    }
    else: {
        400_Bad Request (Invalid OTP code or OTP has expired)
        404_Not Found (No OTP requested for this email)
        422_Validation Error (Missing required fields)
    }
}
```

c) Send Approval OTP (Secure Auth)

URL:/otp/send-approval

Method: POST

Request Body: None

Response: Used when an authority clicks 'Approve' on a dashboard. It automatically sends the OTP to their registered email address.

```
Status:{

    If successful: {
        200_OK (Authorization OTP sent)
    }
    else: {
        401_Unauthorized (User is not logged in)
        500_Internal Server Error (Failed to send the email)
    }
}
```

10. Finances Tab

a) Get My Club Finances

URL: /finances/status

Method: GET

Request Body: None

Response: Fetches the finance ledger (allocated budget, total spent, remaining balance, and transaction history) for the logged-in user's club.

```
Status:{

    If successful: {
        200_OK (Finance ledger retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        404_Not Found (Finance ledger not found for the user's club)
    }
}
```

b) Get Specific Club Finances (Authority Dashboard)

URL: /finances/{club_id}

Method: GET

Request Body: None

```
Response: Fetches the finance ledger of a specific club by its ID.
Restricted to administrative authorities.

Status:{

    If successful: {
        200_OK (Club finance ledger retrieved successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        403_Forbidden (User is not authorized to view other clubs'
finances)
        404_Not Found (Club ledger not found)
        422_Validation Error (Invalid club_id format)
    }
}
```

c) Submit JSON Transaction (Test/API Fallback)

```
URL: /finances/transactions

Method: POST

Request Body (JSON): { "amount": "float", "description": "string" }

Response: Submits a new financial transaction without a file
attachment. Checks if the club has sufficient budget before
processing and updates the ledger.

Status:{
    If successful: {
        200_OK (Transaction successfully recorded)
    }
    else: {
        400_Bad Request (Insufficient budget)
        401_Unauthorized (User is not logged in)
        404_Not Found (Club ledger not found)
        422_Validation Error (Missing required fields)
    }
}
```

d) Upload Bill (Real-World Submission)

```
URL:/finances/upload-bill

Method: POST

Request Body (multipart/form-data): amount: "float" description:
"string" file: [File Upload]
```

```
Response: Submits a new financial transaction with an attached
bill/receipt. Saves the file to the server and updates the club's
finance ledger.

Status:{
    If successful: {
        200_OK (Bill uploaded and transaction recorded successfully)
    }
    else: {
        401_Unauthorized (User is not logged in)
        404_Not Found (Club ledger not found)
        422_Validation Error (Missing required form fields or file)
    }
}
```

3 Completeness

Implemented Features

1. Authentication and Security

- **Two-Step Verification System:** Implemented a secure login mechanism where users verify their password first, followed by a 6-digit OTP (One-Time Password) sent to their registered email address.
- **Privacy and Session Management:** Utilizes JWT (JSON Web Tokens) and role-based access control (RBAC) to ensure users can only access endpoints authorized for their specific roles (e.g., coordinator, gensec, adsa). Passwords are cryptographically hashed in the database.
- **Critical Action Authorization:** OTP verification with a 5-minute expiration is required not just for login, but for authorizing sensitive actions, such as authorities approving venue or MoU requests.

2. Features Implemented in Club Coordinator Section

- **Centralized Request Dashboard:** Coordinators can view a consolidated history and track the real-time pipeline status (Pending, Approved, Rejected) of all their Permission Letters, Venue Bookings, and MoU Requests.
- **Permission Letters:** Coordinators can submit detailed permission letters which automatically enter the administrative approval pipeline. Upon final approval, a unique, trackable Permission Letter ID is generated.
- **Venue Booking System:** Allows coordinators to check real-time room availability to prevent double-booking. To successfully book a venue, the coordinator must provide a valid, fully approved Permission Letter ID belonging to their club.
- **MoU Submissions:** Coordinators can submit Memorandum of Understanding (MoU) requests to the administration and upload the corresponding PDF documents securely.
- **Finance Management Ledger:** Provides a real-time dashboard showing the club's total allocated budget, total spent, remaining balance, and utilization percentage. Coordinators can log transaction details and upload physical bills/receipts for record-keeping.
- **Announcements View:** A dedicated space for coordinators to view official announcements and broadcasts targeted either specifically to their club or to all clubs.

3. Features Implemented in Administrative Authority Section

- **Role-Specific Approval Queues:** Authorities (GenSec, President, FacAd, ADSA, DOSA) have custom dashboards that filter and display only the requests currently pending their specific stage in the approval hierarchy.
- **Strict Multi-Tier Pipeline:** Requests automatically move through a defined pipeline (`Pending GenSec` -> `Pending President` -> `Pending FacAd` -> `Pending ADSA` -> `Approved`).
- **Action Processing:** Authorities can approve or reject requests. Rejections include mandatory comment fields to provide reasoning. Final approval automatically updates the public calendar and dispatches success emails.

- **Club Oversight:** Authorities have the elevated privilege to view the complete request history and detailed financial ledgers of any specific club in the system.
- **Publishing Announcements:** Administrators can publish targeted announcements to specific club coordinators or broadcast messages globally.

4. Features Implemented in Public and Automated Services

- **Public Event Calendar:** A read-only interface that automatically fetches and displays all fully approved upcoming events and their corresponding venue locations.
- **Automated Email Notifications:** An integrated background service that dispatches emails for OTP delivery, new announcements, and real-time status updates whenever a request is approved, rejected, or moved to the next stage in the pipeline.

Future Development Plans

- **Forgot Password and Account Recovery:** Implementing a secure password reset flow. Since the system already utilizes an email service for OTP verification, we will extend this to allow users to securely recover their accounts and reset forgotten passwords via time-sensitive, single-use email links.

- **Club Inventory & Equipment Tracking:** Expanding the database and coordinator dashboard to manage physical assets. Many campus clubs hold expensive equipment. We plan to add a module allowing coordinators to log inventory, track equipment issued to specific students, and monitor return deadlines.

- **Digital Signature Integration:** While MoU and Permission Letter PDFs are currently uploaded manually, we plan to implement a feature that automatically generates the final approved PDF document with embedded digital signatures or stamps from the respective authorities.

- **Advanced Analytics and Reporting Dashboard:** We will introduce a visual analytics dashboard for top-level administrators to view graphical representations of budget utilization across all clubs, as well as venue booking frequency and peak usage times.

- **External Calendar Export:** To make the Public Event Calendar more actionable, we plan to add an "Add to Google Calendar" or iCal export button, allowing users to easily sync approved campus events to their personal devices.

- **Automated Role Handover Mechanism:** Since student and faculty positions change annually, we will implement a streamlined "Role Transfer" feature that safely migrates the dashboard history, remaining budget, and pending requests from an outgoing user to the newly elected official.

User ID (Email)	Password	Role
coord_music@iitk.ac.in	coord_music	Club Coordinator
coord_drams@iitk.ac.in	coord_drams	Club Coordinator
coord_dance@iitk.ac.in	coord_dance	Club Coordinator
coord_fashion@iitk.ac.in	coord_fashion	Club Coordinator
gensec_snt@iitk.ac.in	gensec_snt	GenSec
gensec_anc@iitk.ac.in	gensec_anc	GenSec
gensec_mnc@iitk.ac.in	gensec_mnc	GenSec
president_sg@iitk.ac.in	president_sg	President
facad@iitk.ac.in	FacAd	Faculty Advisor
adsa@iitk.ac.in	adsa	ADSA

dosa@iitk.ac.in	dosa	DOSA

Appendix A - Group Log

Date	Timings	Duration	Minutes
11 Mar	17:00 – 22:00	5 hrs	• Discussed the components to be present and distributed the division of the components.
14 Mar	18:30 – 23:00	4 hrs 30 mins	• Built the individual components and discussed further plans regarding the combination of the components. • Created a base for backend and database.
19 Mar	13:30 – 19:00	5 hrs 30 mins	• Built all the routers of backend and tested using Fastapi swagger. • Revision of the components and distribution of the individual pages of the application.
20 Mar	21:00 – 03:00	6 hrs	• Finished the coordinator dashboard. • Started working on the authority dashboard.
23 Mar	10:00 – 14:00	4 hrs	• Finished authority dashboard. • Finished the approval logic for the frontend. • Started working on the implementation document.
26 Mar	21:00 – 03:00	6 hrs	• Finished the implementation doc. • Finishing up the backend and the integration of the frontend and backend.